057 004

fixtures/mutation_reuse/ — T07 (SpecKit-"fast-dev-cycles", plan T-C)

Fixtures proving `constitution/scripts/fastcycle/gates/mutation_reuse.py` (not yet implemented — T-C07) will correctly implement DEC-23's sound-mutation-reuse rule:

> re-execute a paired mutation only when `hash(patch ‖ target gate script ‖ observed inputs ‖ tool versions)` changed; reuse KILLED only (SURVIVED is always re-examined); detect byte-identical (normalised) mutated scripts and report them as equivalent/duplicate instead of running twice.

Shared fixture content

- `gate_script.sh` / `gate_script_v2.sh` — two genuinely different toy gate scripts (different grep rule) used across scenarios.
- `mutation_patch.diff` / `mutation_patch_v2.diff` — two genuinely different toy mutation patches.
- `observed_input.txt` / `observed_input_flipped.txt` — two byte-differing toy observed-input files.
- `evidence.txt` — a stub evidence file referenced by `--evidence` in the RED test's real tool-invocation section.

Cache-behavior scenarios (put_envelope.json / get_envelope.json / expected)

Each of the 7 directories below carries a `put_envelope.json` (what a prior run stored) and a `get_envelope.json` (what the current run asks about), built from the shared content above, plus an `expected` file whose first line is `HIT` or `MISS` and whose remaining lines explain why:

Scenario	Covers
`mr_good_killed_reuse`	golden-good: unchanged key, KILLED verdict → HIT
`mr_bad_survived_never_reused`	golden-bad: unchanged key, SURVIVED verdict → MISS always
`mr_one_input_changed`	primary case: one observed input byte flipped → MISS
`mr_negctrl_mtime_only`	negative control: mtime-only diff, content same → HIT
`mr_flip_patch`	control needle: different mutation patch → MISS
`mr_flip_gate_script`	control needle: gate script itself changed → MISS
`mr_flip_tool_version`	control needle: toolchain version changed → MISS

Cross-gate isolation scenario (mr_gate_isolation/)

`mr_gate_isolation/` is a distinct fixture shape (its own subdirectory, own `README.md`) from the 7 above: it puts TWO independent gates (`GATE-MR-ISO-A`, `GATE-MR-ISO-B` — distinct gate scripts + distinct mutation patches, so their DEC-23 keys are provably distinct) into ONE SHARED cache directory, closing the "(not others)" half of tasks.md T057's case 1 ("changing one gate's input content re-

runs THAT gate's mutations (not others)") — none of the 7 scenarios above tests this alone, since each of them uses a single hardcoded gate id in its own isolated scratch cache directory. See `mr_gate_isolation/README.md` for the full scenario.

`dec23_key_ref.py` (`../lib/dec23_key_ref.py`) is this project's own reference implementation of the 4-component DEC-23 key formula, used by the RED test's Section B to prove every scenario's put/get envelope pair is non-vacuous (keys genuinely match or genuinely differ, as each scenario claims) BEFORE any claim is made about the absent real tool.

TCE-dedup scenarios (mr_tce_duplicate / mr_tce_distinct)

Two DIRECTORIES, each holding `mutant_a.sh` + `mutant_b.sh` (two real, independently-written toy mutated gate scripts) + an `expected` file whose first line is `DUPLICATE` or `DISTINCT` :

- `mr_tce_duplicate` — the two scripts encode the SAME mutation, differing only in comment placement / blank lines / indentation. After comment/whitespace normalisation they are byte-identical.
- `mr_tce_distinct` — the two scripts encode GENUINELY DIFFERENT mutations (different grep pattern). Normalisation must NOT collapse them (the negative control proving the dedup rule does not over-merge).

`tce_normalize_ref.py` (`../lib/tce_normalize_ref.py`) is this project's own reference normaliser (strip comments outside quotes, collapse whitespace, drop blank lines, SHA-256 the result), self-validated in the RED test before any claim is made about the absent real tool's `check-duplicate` subcommand.

What this RED test does NOT cover

- The mutation cache's PURGE/eviction policy (out of scope for T057; no plan.md text names one for T-C07).
- The interaction between mutation-reuse and the flake ledger (T-C09, `flake_ledger.py`) — DEC-23 states mutations are "sampling/prediction confined to the nightly drift lane (T-C10)" but does not name a specific flake-

and-reuse interaction fixture; left to T-C09's own test (`test_flake_ledger_red.sh`, T059) per this project's `producer != oracle` scope-separation discipline (§11.4.240) — this file's scope is DEC-23's reuse/dedup rule alone.

- The CLI wire format below is UNCONFIRMED by any contracts/*.md file (none exists for T-C07 at the time this file was written — confirmed by a real directory listing, not assumed) and is DEFINED here, binding-if-adopted, per this project's established house precedent (test_verdict_cache_red.sh's header comment; fixtures/verdict_cache/README.md).